LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load

Pranay Tummalapalli
Conscious Engines
pranay@consciousengines.com

Sahil Arayakandy
Conscious Engines
sahil@consciousengines.com

Ritam Pal
Conscious Engines
ritam@consciousengines.com

Kautuk Kundan
Conscious Engines
kautuk@consciousengines.com

Abstract

Deploying large language models on-device for always-on personal agents demands sustained inference from hardware tightly constrained in power, thermal envelope, and memory. We benchmark Qwen 2.5 1.5B (4-bit quantised) across four platforms spanning the performance–efficiency spectrum: a Raspberry Pi 5 with Hailo-10H NPU, a Samsung Galaxy S24 Ultra, an iPhone 16 Pro, and a laptop NVIDIA RTX 4050 GPU. Using a fixed 250-token prompt over 20 warm-condition iterations per device, we measure throughput, latency, power, and thermal behaviour. Our results show that thermal management, not peak compute, is the binding constraint for sustained mobile inference: the iPhone 16 Pro loses nearly half its throughput within two iterations and the S24 Ultra suffers a hard OS-enforced GPU frequency floor that terminates inference entirely. The RTX 4050 sustains 131.7 tok/s at 34.1 W with no throttling. In contrast, the Hailo-10H NPU sustains 6.9 tok/s at under 2 W with near-zero performance variance, matching the RTX 4050 in energy proportionality at $19\times$ lower throughput. These findings carry direct implications for always-on agent design: smartphones are viable only for intermittent inference at low query rates, while dedicated edge NPUs offer thermally stable, power-proportional deployment at the cost of throughput. For latency-tolerant, continuously available agents, purpose-built NPU hardware represents a more dependable deployment substrate than mobile SoCs operating under passive thermal constraints.

1 Introduction

The emergence of sub-2B parameter language models capable of useful instruction following [1, 2] has renewed interest in on-device LLM deployment. Running inference locally offers offline capability, reduced latency, and privacy, properties that are particularly valuable for always-on personal agents that must respond to continuous streams of user queries without round-tripping to a cloud server.

Modern mobile SoCs include dedicated neural processing units with tens of TOPS of claimed throughput [5, 6], and quantisation techniques such as GPTQ [3] and AWQ [4] compress models to under 1 GB, making them memory-feasible on phones and single-board computers. Yet the gap between peak hardware specifications and *sustained* real-world inference performance under thermal

Preprint. Under review.

load remains poorly characterised. Existing benchmarks either target single-inference latency [8, 13], cloud-hosted throughput [7], or report thermal effects only as a side observation rather than a primary focus [9]. The MELTing Point framework [9] provides the closest prior work, characterising LLM inference on mobile devices with attention to energy and thermal behaviour, but does not run extended back-to-back query sequences sufficient to reveal the full thermal degradation curve. Xiao et al. [10] benchmark LLMs on mobile platforms with attention to throughput and observe 10–20% degradation over five consecutive runs, but do not characterise steady-state thermal floors or test beyond a handful of iterations. MLPerf Inference [7] now includes edge LLM scenarios as of v5.1, but measures peak performance without tracking thermal throttling or sustained-workload behaviour.

On the hardware side, quantised LLM inference on dedicated edge NPUs remains largely unexplored in the literature. To the best of our knowledge, no independent benchmark of LLM inference on Hailo hardware has been published; available figures derive from vendor publications alone.

This leaves three questions unanswered: how mobile inference performance degrades over repeated consecutive queries; whether low-power NPU accelerators can match mobile devices in sustained throughput per watt; and what the practical trade-offs are between a dedicated GPU, a mobile SoC, and an edge NPU for always-on agent deployments.

To answer these, we conduct controlled warm-condition benchmarking of Qwen 2.5 1.5B across four platforms, a Raspberry Pi 5 with Hailo-10H NPU, a Samsung Galaxy S24 Ultra, an iPhone 16 Pro, and a laptop with NVIDIA RTX 4050 GPU, using a standardised prompt and uniform metric collection across 20 iterations per device. Our principal contributions are: an independent cross-platform benchmark of sustained LLM inference including a dedicated edge NPU; empirical characterisation of thermal degradation curves on two flagship smartphones showing qualitatively different failure modes; and evidence that the Hailo-10H NPU achieves thermally stable, near-zero-variance inference at energy proportionality comparable to a laptop GPU, positioning dedicated edge NPUs as a compelling platform for always-on, power-sensitive deployments. However, the Hailo-10H's current throughput of 6.9 tok/s, constrained single-sequence autoregressive decode characteristics, represents a significant latency limitation for interactive applications, and should be interpreted as a platform deployment ceiling rather than a fundamental bound on NPU-class inference.

2 Background

2.1 Quantised Inference on Edge Hardware

Post-training quantisation reduces weight precision from BF16/FP16 to INT4, yielding approximately $4\times$ model size reduction with minimal perplexity degradation on instruction-tuned models [3, 4]. Q4_0 (4-bit grouped quantisation) has become the dominant format for on-device deployment due to its compatibility with GGUF, MLC-LLM, and MLX runtimes, while GPTQ [3] provides an alternative calibrated quantisation path used by vLLM and PyTorch-based serving stacks. llama.cpp [12] provides cross-platform GGUF-format inference across targets.

2.2 Autoregressive Decoding Phases

LLM inference comprises two phases with distinct compute characteristics. During *prefill*, the input prompt is processed in parallel; this phase is compute-bound, proportional to prompt length, and dominates time-to-first-token (TTFT). During *decode*, tokens are generated one at a time autoregressively; this phase is memory-bandwidth-bound, with throughput determined by KV-cache access speed. Peak hardware TOPS ratings reflect throughput under compute-bound, high-utilisation conditions. Prefill is relatively compute-bound and scales with available TOPS; sustained decode throughput is memory-bandwidth-bound, with throughput determined by KV-cache access speed, a key distinction when evaluating always-on agents that generate long responses.

2.3 Thermal Throttling in Mobile SoCs

Mobile SoCs rely on passive cooling and must respect strict power budgets to avoid skin temperature violations. When junction temperatures exceed thermal thresholds, Dynamic Voltage and Frequency Scaling (DVFS) reduces clock frequencies, directly reducing throughput. Sustained LLM inference is particularly susceptible to this mechanism. Recent work on mobile DVFS governors [14] has shown

that Energy-Aware Scheduling can cause CPUs to run at suboptimally low frequencies during LLM inference, degrading latency and energy efficiency, illustrating that OS-level power management interacts with inference workloads in ways beyond simple thermal capping.

3 Methodology

3.1 Platforms and Inference Stacks

We select four platforms spanning the performance–efficiency spectrum relevant to personal agent deployment: a Raspberry Pi 5 paired with a Hailo-10H M.2 NPU module, representing always-on low-power edge deployment; a Samsung Galaxy S24 Ultra as a flagship Android device with Snapdragon 8 Gen 3; an iPhone 16 Pro as a flagship iOS device with Apple A18 Pro; and a laptop with NVIDIA RTX 4050 GPU as a battery-powered edge device in a laptop form factor. Full hardware and software specifications are given in Tables 1 and 2.

The Hailo-10H connects via PCIe Gen 3.0, providing 40 TOPS at under 5 W. hailo-ollama partitions model layers between NPU and CPU, with attention layers offloaded to the NPU and Q4 weights utilising the NPU's INT8 compute units via the Hailo Dataflow Compiler. Power is estimated from the RPi 5 PMIC's `VDD_CORE_A` current and `VDD_CORE` and `EXT5V_V` voltage rails sampled via the INA219 sensor at 1 kHz. This measurement reflects total system draw rather than isolated NPU power; while `EXT5V_V` is the primary supply rail for the Hailo-10H over PCIe, the absence of a dedicated current sensor on that pin means the reported figures are indicative of NPU power consumption but cannot be attributed to the accelerator in isolation.

The Samsung Galaxy S24 Ultra runs Qwen 2.5 1.5B compiled to TVM binary format targeting the Adreno 750 GPU via Apache TVM 0.14 through MLC-LLM. The benchmark runs as a custom headless Android service (`BenchmarkService`) on Android 16, with per-iteration metrics logged via a terminal-style `LogActivity` interface. Power measurement via the Android Battery Manager API was found unreliable under active GPU load and is not reported (see Section 5).

The iPhone 16 Pro runs a custom SwiftUI benchmarking application (`MLXBenchmark`) integrating MLX Swift (v0.29.1) with inference dispatched to the A18 Pro GPU via Metal compute kernels; the Neural Engine is not utilised by MLX. Thermal state is monitored via `ProcessInfo.thermalState` and battery level via `UIDevice.current.batteryLevel` at each iteration boundary. iOS does not expose per-component power draw to third-party applications, so battery state-of-charge is used as the sole energy proxy.

The RTX 4050 laptop serves the model via vLLM on Ubuntu with PyTorch backend and CUDA 12.1. GPU power, utilisation, and temperature are logged via `nvidia-smi` at 100 ms intervals. All runs were conducted on battery under OS power management, which imposes a soft sustained power ceiling near 35 W, approximately 45% of the GPU's rated 75 W TGP. Reported throughput therefore represents battery-throttled performance.

Table 1: Hardware specifications across evaluated platforms.

	RPi 5 + Hailo-10H	**S24 Ultra**	**iPhone 16 Pro**	**RTX 4050 Laptop**
SoC / CPU	BCM2712 4×A76 @ 2.4 GHz	Snapdragon 8 Gen 3	Apple A18 Pro	Intel Core i7-13700H
AI Accel.	Hailo-10H NPU (40 TOPS, <5 W)	Hexagon NPU (45 TOPS)	Neural Engine 17-core (∼35 TOPS)	RTX 4050 (80 Tensor Cores)
RAM	8 GB LPDDR4X	12 GB LPDDR5X	8 GB Unified	32 GB DDR5
GPU	VideoCore VII	Adreno 750	6-core Apple GPU	RTX 4050 (2560 CUDA cores)
VRAM	Shared	Shared	Shared	6 GB GDDR6
OS	Raspberry Pi OS	Android 16	iOS 26	Ubuntu 22.04.3
Idle power	∼3.5 W	∼0.8 W	∼0.6 W	∼15 W[§]
Max power	∼12 W	∼12 W	∼10 W	75 W TGP[†]

[†]Acer Nitro V RTX 4050 laptop GPU TGP is 75 W (60 W base + 15 W Dynamic Boost). Benchmarks conducted on battery; sustained inference power was observed at ∼34 W.

[§]System idle power ∼15 W; GPU idle power ∼2 W per `nvidia-smi`.

Table 2: Software stack versions across platforms.

	RPi 5 + Hailo-10H	**S24 Ultra**	**iPhone 16 Pro**	**RTX 4050 Laptop**
Framework	hailo-ollama	MLC-LLM 0.1.0	MLX Swift v0.29.1	vLLM (CUDA 12.1)
Backend	HailoRT 4.17.0	OpenCL 3.0 + TVM 0.14	Metal (MLX)	PyTorch 2.1.0
Language	Python 3.10	Java / NDK r26b	Swift 5.9 (SwiftUI)	Python 3.10
Model format	GGUF Q4_0	MLC binary q4f16_2	MLX safetensors Q4_0	GPTQ Int4 safetensors
Context window	32,768	32,768	32,768	32,768
Power monitor	INA219 (1 kHz)	Battery API (100 ms)*	`UIDevice` batteryLevel	`nvidia-smi` (100 ms)

*Android Battery Manager API power figures were found unreliable under active GPU load and are excluded from analysis.

3.2 Model

We select Qwen 2.5 1.5B [1] with 4-bit quantisation as the benchmark model. Selection criteria were threefold: native support across all four inference frameworks, a sub-1 GB memory footprint fitting all device constraints, and a uniform quantisation level across platforms to minimise compression as a confounding variable. Each platform uses a native format conversion of the same base weights: GGUF (hailo-ollama), TVM binary (MLC-LLM), MLX safetensors (iPhone 16 Pro), and HuggingFace safetensors (vLLM). Weight equivalence is verified via SHA-256 checksums on quantised tensors. Key specifications are listed in Table 3.

Table 3: Qwen 2.5 1.5B model specifications and per-platform format details.

Attribute	**Value**
Parameters	1.5B
Architecture	Transformer decoder, GQA (2 groups)
Layers	28
Hidden size	1536
Attention heads	12
Vocabulary	151,936 tokens
Context window	32,768 tokens

3.3 Prompt and Generation Configuration

All platforms receive an identical prompt (UTF-8, 250 tokens) that elicits long-form structured output, imposing sustained decode load to stress thermal management and memory-bandwidth utilisation rather than burst performance:

```
Write an in-depth, structured, and self-contained essay explaining the
concept of consciousness from multiple perspectives. Begin by clearly
defining consciousness and why it is a difficult concept to study. Then
explore the topic from the following viewpoints, dedicating multiple
detailed paragraphs to each:

  1. Philosophical perspectives, including classical and modern views,
     major debates, and unresolved questions.
  2. Neuroscientific perspectives, covering brain structures, neural
     correlates, current theories, and experimental approaches.
  3. Cognitive science and psychology perspectives, including perception,
     attention, self-awareness, and consciousness disorders.
  4. Artificial intelligence and machine consciousness, discussing whether
```

```
    machines can be conscious, functional vs. phenomenal consciousness,
    and current limitations of AI systems.
  5. Ethical and societal implications of understanding or engineering
    consciousness.

Throughout the essay: use clear section headings; explain all technical
terms in plain language; provide illustrative examples where helpful;
compare and contrast different viewpoints; avoid bullet lists unless
necessary for clarity. Conclude with a reflective summary discussing what
remains unknown and what future research directions may look like. The
response should be detailed, continuous, and written in a neutral,
academic tone.
```

Token generation is capped at 1,000 tokens on iOS; all other platforms are uncapped (EOS-terminated).

3.4 Metrics

Table 4 defines all metrics collected during benchmarking. Not all metrics are available on all platforms: thermal state is iOS-only, GPU frequency is Android-only, and power and energy-per-token are reported only for the RTX 4050 and Hailo-10H due to measurement limitations on mobile platforms discussed in Section 5.

Table 4: Metrics collected across platforms.

Metric	Unit	Definition
Decode tokens	count	Tokens generated during the decode phase
Decode time	ms	Wall-clock duration of the decode phase
Prefill Time	ms	Time from prompt submission to first output token; equivalent to the duration of the prompt encoding phase
Throughput (TPS)	tok/s	$N_{\text{decode}}/t_{\text{decode}}$
Avg power	W	Mean system power draw over inference duration
Peak power	W	Maximum instantaneous power observed
Energy/token	mJ	$P_{\text{avg}} \cdot t_{\text{decode}}/N_{\text{decode}}$
CPU / GPU temp	°C	Maximum temperature recorded during inference
Thermal state	—	iOS categorical signal: Normal, Warm, or Hot
Battery drain	% SoC	Change in state of charge over full benchmark run
GPU frequency	MHz	Observed GPU clock frequency per iteration (Android only)

3.5 Experimental Protocol

We focus on warm-condition inference: the model is loaded once and remains in memory across all iterations, representing realistic interactive assistant usage where queries arrive consecutively. Cold-start model loading time is recorded where observable but excluded from throughput and latency analysis.

Before each benchmark session, the device is allowed to equilibrate for 10 minutes at ambient temperature (22°C $\pm$ 2°C). After loading the model and discarding one warm-up inference, we verify thermal stability ($\Delta T < 2$°C over 60 seconds) before executing 20 iterations with a 1-second inter-iteration gap. Per-iteration results are exported to CSV and validated for token count anomalies.

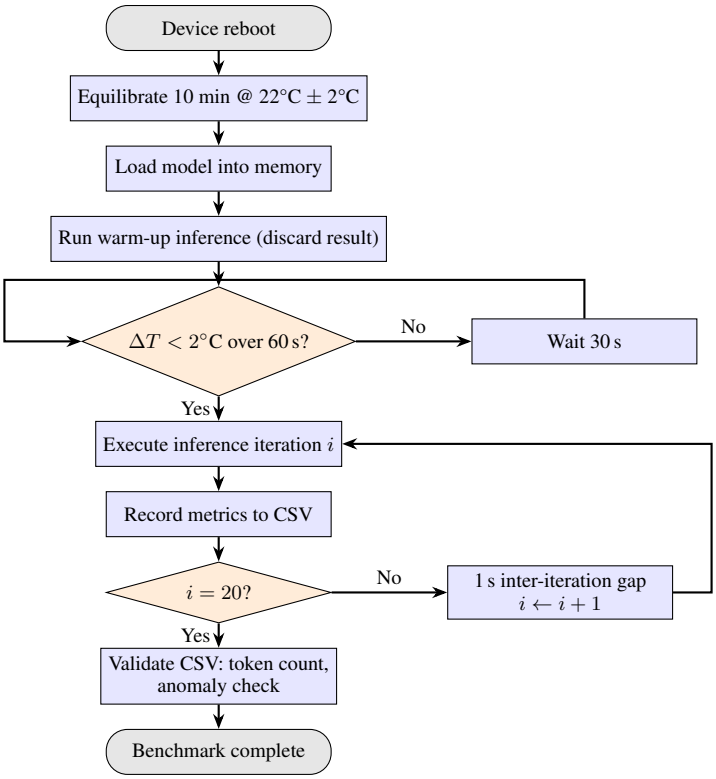

Figure 1: Experimental protocol flowchart. Each platform undergoes this procedure independently. Cold-start prefill-time from the warm-up inference is recorded separately and excluded from throughput analysis.

4 Results

4.1 NVIDIA RTX 4050 (Desktop GPU)

The RTX 4050 provides the performance baseline in this study, representative of a battery-powered edge device in a laptop form factor. Table 5 summarises results across 20 warm iterations; run 1 includes cold-start overhead and is excluded from steady-state statistics.

Table 5: RTX 4050 inference performance (runs 2–20, warm condition). Power values derived from system-level measurement at 100 ms polling.

Metric	**Mean**	**Std Dev**	**Min**	**Max**
Decode tokens	1789	0	1789	1789
Decode time (ms)	13,697	274	12,881	13,906
Throughput (tok/s)	131.70	2.87	128.65	138.88
Prefill time (ms)	1,998	42	1,894	2,028
Avg system power (W)	34.12	0.17	33.75	34.40
Peak system power (W)	35.28	0.21	35.05	35.73
Energy per token (mJ)	297.3	6.5	282.7	304.9
GPU temperature (°C)	65.4	4.3	55	70
CPU temperature (°C)	87.5	4.2	77	93
Battery drain	12% over 20 runs (100% → 88%)			

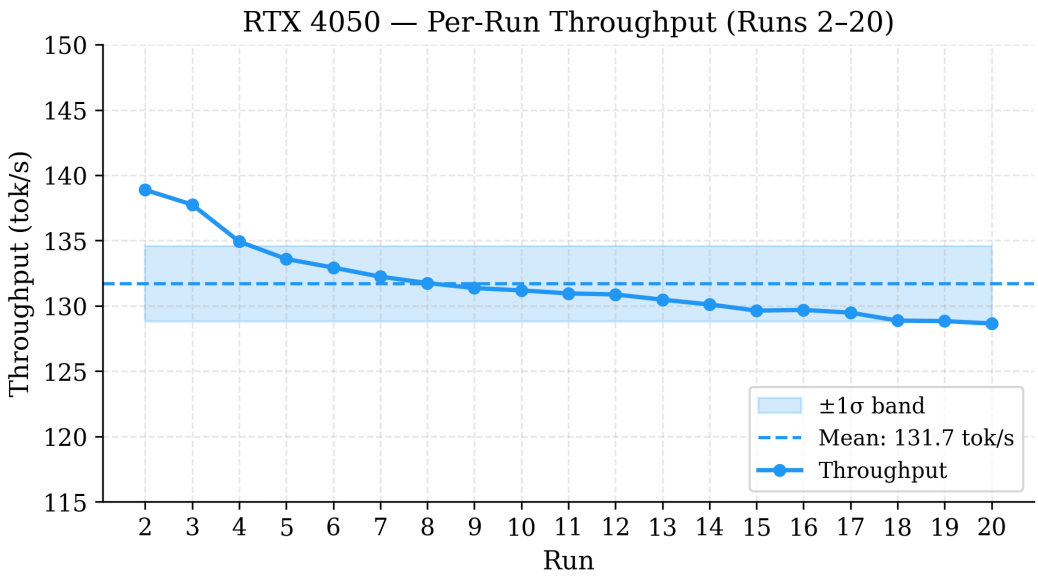

Figure 2: RTX 4050 per-run throughput (runs 2–20). Dashed line marks mean of 131.70 tok/s. CV of 2.2% confirms stable battery-throttled performance.

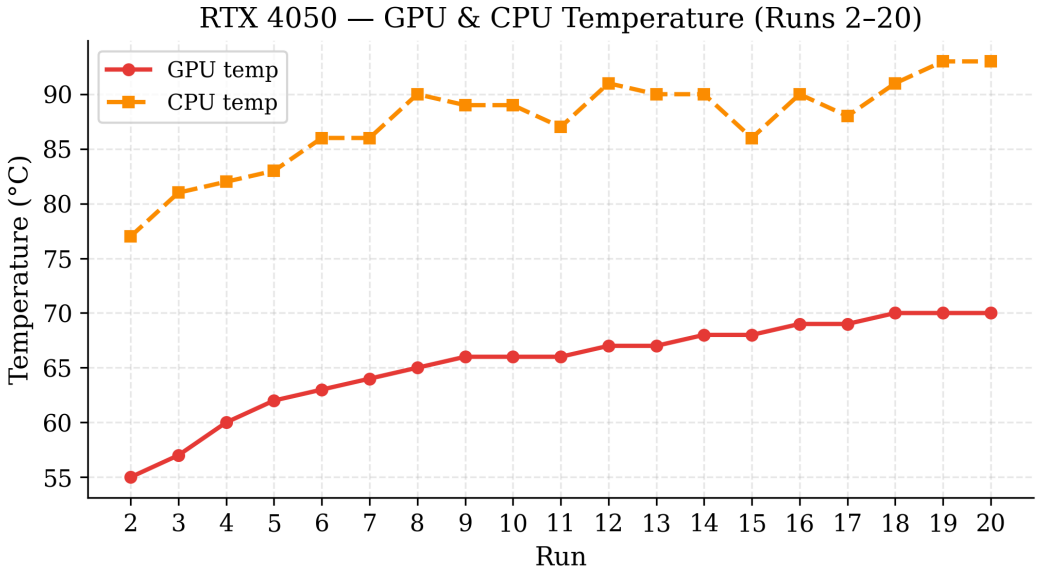

Figure 3: RTX 4050 GPU and CPU temperature across runs 2–20. GPU rises from 55°C to 70°C; no throttling observed.

The RTX 4050 sustains a mean throughput of 131.7 tok/s ($\sigma = 2.87$, CV = 2.2%) with deterministic output length (1789 tokens per run). Run 1 is excluded as it reflects cold-start model loading (93.8 tok/s, 52.8 W average power). Under battery power, average power stabilises at 34.1 ± 0.2 W with transient peaks reaching 35.73 W, confirming the workload is memory-bandwidth bound rather than compute bound. GPU temperature rises from 55°C at run 2 to 70°C by run 20, reflecting gradual heat buildup, though no thermal throttling is observed—the throughput CV of 2.2% confirms stable performance throughout. Energy per token stabilises at 297 ± 7 mJ across warm runs.

4.2 Raspberry Pi 5 + Hailo-10H NPU

Table 6 summarises results across 20 runs. Run 1 includes a cold-start prefill-time of 11.86 s reflecting initial model loading into NPU memory and is excluded from steady-state statistics.

Table 6: RPi 5 + Hailo-10H inference performance (runs 2–20, warm condition).

Metric	Mean	Std Dev	Min	Max
Decode tokens	564	0	564	564
Decode time (ms)	81,569	34	81,516	81,640
Throughput (tok/s)	6.914	0.003	6.908	6.919
Prefill time (ms)	1,287	1	1,286	1,290
Avg system power (W)	1.870	0.016	1.851	1.921
Peak system power (W)	3.065	1.099	2.383	6.662
Energy per token (mJ)	270.5	2.4	267.6	277.9
CPU temperature (°C)	52.7	0.8	51.0	53.8
NPU temperature (°C)	58.5	1.1	55.9	60.0

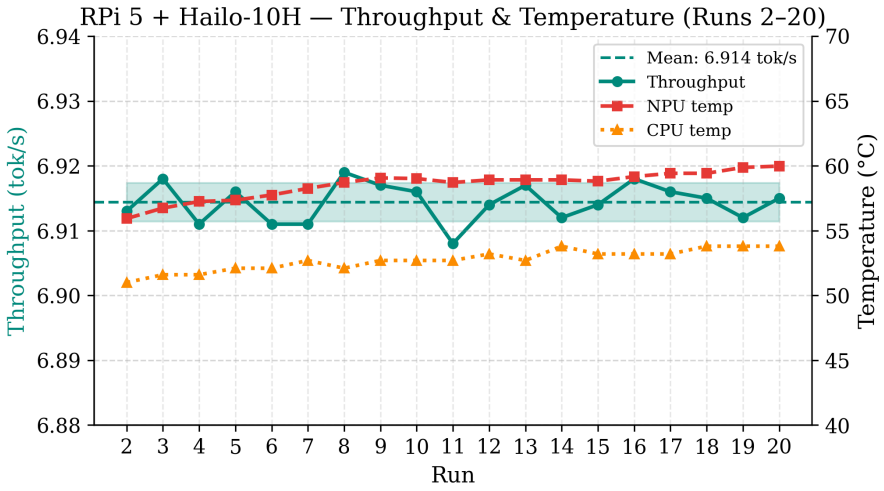

Figure 4: RPi 5 + Hailo-10H throughput (bars, left axis) and NPU/CPU temperature (lines, right axis) across runs 2–20. Throughput CV of 0.04% and stable temperatures confirm no throttling at any point.

The Hailo-10H delivers the most consistent inference of any platform in this study. With greedy decoding, output length is fully deterministic, all 19 warm runs produce exactly 564 tokens, yielding a decode time CV of 0.04% and a throughput CV of 0.04%, effectively zero variance. Prefill is similarly invariant at $1{,}287 \pm 1$ ms (CV = 0.11%), reflecting the deterministic dispatch behaviour of the NPU hardware. Average system power (RPi 5 + Hailo-10H combined) is 1.87 ± 0.02 W, with energy per token of 271 ± 2 mJ, the lowest of any platform in this study. Thermal behaviour is exemplary: CPU temperature is confined to 52.7 ± 0.8°C and NPU temperature to 58.5 ± 1.1°C, with no upward trend and no throttling observed at any point across all 20 iterations.

The observed throughput of 6.914 tok/s is substantially below the Hailo-10H's rated 40 TOPS peak compute figure, which is expected given the nature of autoregressive LLM decode. Vendor TOPS ratings reflect peak throughput under idealised batch workloads; sustained single-sequence decode is memory-bandwidth bound rather than compute bound, and cannot exploit the NPU's parallel compute units efficiently. The result is broadly consistent with Hailo's own published figure of 6.82 tok/s for Qwen2.5-1.5B-Instruct on the same hardware configuration, suggesting the deployment is operating near the practical ceiling for this platform. The primary bottleneck is on-module LPDDR4 memory bandwidth rather than PCIe: although the Hailo-10H exposes a PCIe Gen 3 ×4 interface (4 GB/s

theoretical), the Raspberry Pi 5 provides only a PCIe Gen 2 ×1 link (500 MB/s theoretical, 400 MB/s effective), this constraint does not appear to be the binding limiter given that our measured throughput matches Hailo's reference figure obtained under identical link conditions. Additional deployment-specific constraints include CPU–NPU layer partitioning in hailo-ollama, where attention layers not offloaded to the NPU execute on the ARM Cortex-A76, and per-token HailoRT dispatch overhead over PCIe. The 6.914 tok/s result should therefore be interpreted as a memory-bandwidth-bound deployment figure reflecting the practical ceiling of this platform, rather than a reflection of the NPU's peak compute capability.

4.3 iPhone 16 Pro (iOS / MLX)

Table 7 reports available metrics across 20 iterations. Per-inference power draw and energy-per-token are not reported for this platform due to iOS measurement limitations.

Table 7: iPhone 16 Pro inference performance (20 iterations, warm condition).

Metric	Mean	Std Dev	Min	Max
Decode tokens	885	142	601	1000
Decode time (ms)	37,136	9,213	17,820	48,221
Throughput (tok/s)	24.75	5.14	20.74	40.35
Battery drain (total)	10% over 20 iterations (100% → 90%)			
Thermal state	Normal (iter 1–2), Warm (3–7), Hot (8–20)			

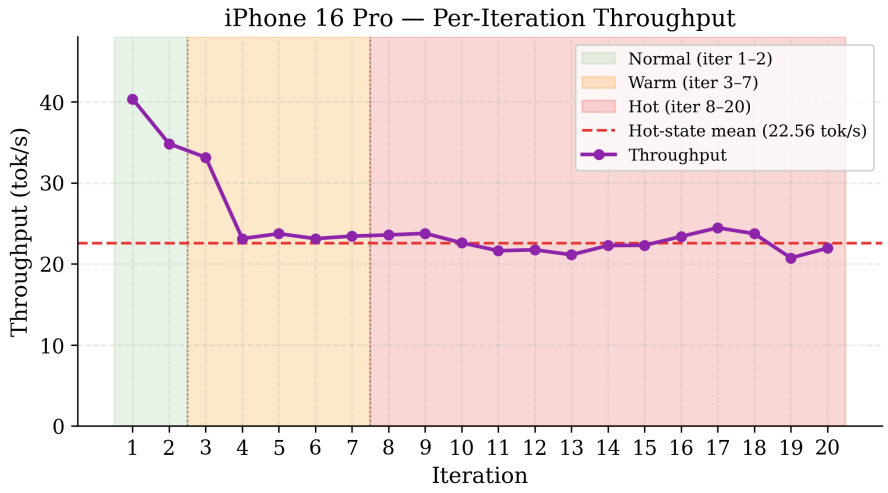

Figure 5: iPhone 16 Pro per-iteration throughput across 20 iterations. Bar shading indicates thermal state: light (Normal, iter 1–2), hatched (Warm, iter 3–7), dark (Hot, iter 8–20). Dashed line marks the sustained Hot-state mean of 22.56 tok/s.

The iPhone 16 Pro exhibits a clear three-phase thermal trajectory over 20 iterations. During the first two iterations (Normal state), mean throughput is 37.58 tok/s, peaking at 40.35 tok/s. By iteration 3 the device transitions to a Warm state (25.31 tok/s, -37.3% from peak), and from iteration 8 onwards enters a sustained Hot state (22.56 ± 1.14 tok/s, -44.1% from peak) that persists for the remaining 13 iterations, 65% of the benchmark. The overall throughput CV of 20.8% is driven almost entirely by this thermal state transition rather than run-to-run noise; within the Hot state alone, CV drops to 5.0%, comparable to the other platforms. The 1-second inter-iteration cooldown is insufficient to permit any thermal recovery. Battery state of charge drops 10% over 20 iterations, projecting to approximately 200 inferences per full charge.

The observed thermal degradation is consistent with prior work on iOS LLM inference. Independent benchmarking of Apple devices under sustained inference [10] finds that Apple GPU performance un-

der iOS exhibits noticeable fluctuations throughout the inference process, with degradation occurring earlier and becoming more pronounced under long-prompt workloads, consistent with our 250-token prompt driving transition to the Hot thermal state by iteration 8. A separate study on iPhone LLM inference [18] explicitly places devices in an ice-cooled environment to mitigate thermal throttling, noting that real-world deployment conditions induce heat buildup and frequency scaling not captured under controlled thermal conditions, corroborating our finding that the 1-second inter-iteration gap is insufficient for thermal recovery under natural operating conditions.

4.4 Samsung Galaxy S24 Ultra (Android / MLC-LLM)

The Android benchmark yielded only 6 iterations before the OS intervened. From iteration 6 onwards, the Snapdragon 8 Gen 3's GPU frequency was floored by the Android thermal governor to 231 MHz, down from 629–680 MHz in prior iterations, rendering further inference non-representative. Only iterations 1–5 are used for statistical analysis; iteration 6 is reported separately to document the throttling event.

Table 8: Samsung S24 Ultra inference performance (iterations 1–5, warm condition). Iteration 6 excluded from summary statistics due to OS GPU frequency floor event.

Metric	Mean	Std Dev	Min	Max
Decode tokens	504	66	419	569
Decode time (ms)	51,294	9,634	36,944	60,274
Throughput (tok/s)	9.93	0.79	9.44	11.34
Prefill time (ms)	25,128	512	24,417	25,733
GPU temperature (°C)	63.0	1.4	61.9	65.4
CPU temperature (°C)	61.1	1.7	59.7	64.0
Battery drain (total)	1% over 5 iterations (94% → 93%)			
Iteration 6 (GPU freq floor event, excluded): TPS = 8.77, GPU freq = 231 MHz, Max GPU temp = 78.3°C, Max CPU temp = 73.8°C				

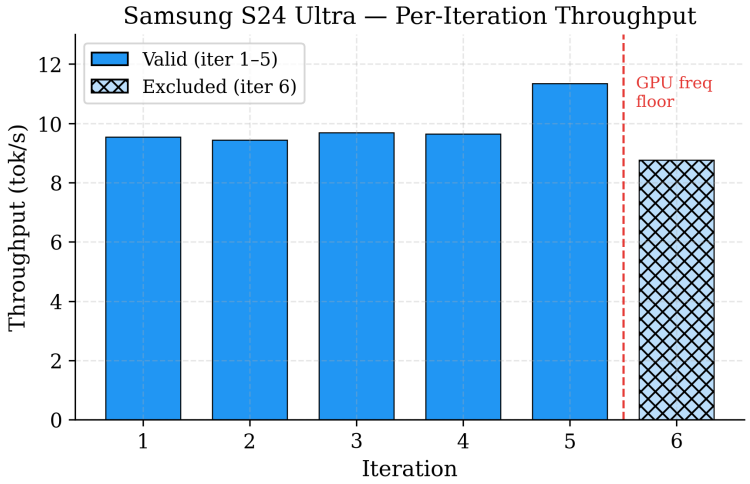

Figure 6: Samsung S24 Ultra per-iteration throughput. Valid iterations 1–5 (dark) are used for analysis. Iteration 6 (hatched, light) is excluded: the Android thermal governor floored GPU frequency to 231 MHz, with GPU temperature reaching 78.3°C.

Across the five valid iterations, the S24 Ultra sustains a mean throughput of 9.93 ± 0.79 tok/s (CV = 8.0%) with a consistent prefill time of $25{,}128 \pm 512$ ms (CV = 2.0%), reflecting stable Adreno 750 GPU scheduling prior to throttling. The anomalously high prefill-time relative to other platforms is likely attributable to MLC-LLM's OpenCL compilation and dispatch overhead on the Adreno 750

rather than raw hardware latency. Iteration 6 documents a hard OS-enforced thermal failure: GPU temperature reached 78.3°C and CPU temperature 73.8°C, the highest thermal readings recorded across all platforms in this study, and no further iterations completed. This abrupt frequency floor, in contrast to the iPhone 16 Pro's gradual DVFS-based degradation, represents a qualitatively different and more disruptive failure mode for sustained inference.

The observed figures are broadly consistent with prior work on Snapdragon 8 Gen 3 inference. Transformer-Lite [17] reports decode throughput of 10–20 tok/s on Snapdragon 8 Gen 3 for Qwen 1.5 4B under MLC-LLM, suggesting our 9.93 tok/s result for the smaller 1.5B model is plausible and likely constrained by MLC-LLM's OpenCL backend efficiency rather than raw hardware capability. Prior work on mobile LLM benchmarking [10] specifically notes that MLC-LLM on Adreno GPUs performs worse in prefill speed than llama.cpp on CPU, attributed to the difficulty of optimising LLM operators across heterogeneous mobile GPU architectures, consistent with our anomalously high prefill-time of 25,128 ms, which we attribute to MLC-LLM OpenCL kernel compilation and dispatch overhead rather than hardware prefill latency. The early thermal termination at iteration 6 is also consistent with known Snapdragon thermal behaviour: independent mobile benchmarking studies [10] limit inference to five consecutive runs per session with mandatory reboots between rounds to avoid overheating, corroborating our observation that sustained back-to-back inference without thermal recovery is not viable on this platform.

4.5 Cross-Platform Comparison

Table 9 summarises sustained inference performance across all four platforms, and Table 10 compares power efficiency for the two platforms with reliable power measurements.

Table 9: Cross-platform sustained inference performance summary. iPhone figures reflect sustained Hot state (iterations 8–20). Android figures reflect valid iterations 1–5 only. †Benchmark conducted on battery; throughput reflects battery-throttled performance.

Platform	TPS (sustained)	TPS CV	Avg Power (W)	Energy/Token (mJ)
RTX 4050†	131.70 ± 2.87	2.2%	34.12	297.3
iPhone 16 Pro (Hot state)	22.56 ± 1.14	5.1%	—	—
S24 Ultra	9.93 ± 0.79	8.0%	—	—
RPi 5 + Hailo-10H	6.914 ± 0.003	0.04%	1.870	270.5

Table 10: Power and energy efficiency comparison (platforms with reliable power measurement only).

Platform	Avg Power (W)	TPS	W/tok/s	Energy/Token (mJ)
RTX 4050†	34.12	131.70	0.259	297.3
RPi 5 + Hailo-10H	1.870	6.914	0.271	270.5

The RTX 4050 leads by a substantial margin at 131.7 tok/s, delivering $19.0\times$ the throughput of the Hailo-10H, $5.8\times$ that of the iPhone 16 Pro in its sustained throttled state, and $13.3\times$ that of the S24 Ultra. Among mobile platforms, the iPhone 16 Pro outperforms the S24 Ultra by $2.3\times$ in sustained throughput, though direct comparison is complicated by differing inference frameworks and the S24 Ultra's premature benchmark termination.

The Hailo-10H achieves a throughput CV of 0.04%, two orders of magnitude lower than any other platform, a consequence of deterministic NPU scheduling and fixed-length greedy decoding. Among platforms with reliable power measurements, the Hailo-10H and RTX 4050 exhibit near-identical energy proportionality: 0.271 and 0.259 W per tok/s respectively. Despite an $18.2\times$ difference in absolute power draw and a $19\times$ difference in throughput, the two platforms deliver equivalent computation per joule, with energy per token of 270.5 mJ (Hailo-10H) versus 297.3 mJ (RTX 4050).

5 Discussion

5.1 Thermal Management as the Binding Constraint

Our results provide strong empirical support for the claim that thermal management, not peak compute, is the binding constraint for sustained mobile LLM inference. The iPhone 16 Pro demonstrates excellent peak throughput (40.35 tok/s) but cannot sustain it: degradation occurs within two inferences and the device remains in a throttled state for 65% of the benchmark, settling at 22.56 tok/s, a 44% reduction. This is consistent with the passive-cooling design of smartphones, where junction-to-ambient thermal resistance is high and heat dissipation is constrained by form factor.

The S24 Ultra exhibits a qualitatively different and more severe failure mode. Rather than gradual DVFS-based throughput reduction, the Android thermal governor enforces a hard GPU frequency floor at iteration 6, causing GPU temperature to reach 78.3°C and terminating the benchmark entirely. For agent deployments, this is more disruptive than graceful degradation: the system becomes unusable for inference rather than merely slower. The 1-second inter-iteration gap does not permit thermal recovery on either mobile platform, suggesting that always-on agents requiring queries more frequent than several minutes apart would face sustained degradation or risk throttle-induced termination.

5.2 The Case for Dedicated Edge NPUs

The Hailo-10H demonstrates that purpose-built NPU inference is thermally stable at a fraction of mobile platform power draw. At 6.914 tok/s and 1.87 W, it delivers energy proportionality comparable to the RTX 4050 despite operating at $19\times$ lower throughput. The platform exhibits near-zero performance variance, deterministic prefill time, and a power envelope compatible with battery or low-wattage PSU operation. For proactive agent scenarios, where responses are generated asynchronously and consistent availability is more critical than low latency, this trade-off is compelling. The cold-start model loading cost (11.86 s) is a one-time overhead that does not affect steady-state operation.

The principal limitation of the current deployment is throughput: at 6.914 tok/s, response generation for a 500-token reply requires approximately 72 seconds, acceptable for background summarisation or scheduled digest generation, but prohibitive for interactive turn-by-turn dialogue. This constraint is, however, substantially attributable to the deployment configuration rather than the NPU itself. Further gains are plausible through batched or speculative decoding strategies adapted for NPU dispatch. The 6.914 tok/s figure should therefore be read as a conservative lower bound on Hailo-class NPU inference rather than a ceiling, and the thermal and efficiency properties demonstrated here are likely to persist as throughput improves with better host integration.

5.3 Deployment Implications

Table 11 maps our empirical findings to deployment scenarios. Our data suggest a practical threshold: mobile devices can support intermittent queries at approximately 5–10 per hour before thermal constraints dominate. Beyond this rate, dedicated edge NPU hardware or tethered GPU systems are required. The Hailo-10H is the only platform in this study that satisfies all three requirements of an always-on agent: thermal stability, low power draw, and consistent availability across sustained workloads. The trade-off for this stability is throughput: at 6.914 tok/s, the Hailo-10H delivers $19\times$ lower throughput than the battery-throttled RTX 4050 and $3.3\times$ lower than the iPhone 16 Pro in its sustained Hot state, a significant constraint for latency-sensitive applications requiring rapid response generation.

Table 11: Deployment scenario suitability based on empirical results. $\sim$ = marginal; $\times$= unsuitable; † = terminated at iteration 6 due to GPU frequency floor.

Scenario	RTX 4050	iPhone 16 Pro	S24 Ultra	RPi + Hailo
Interactive assistant (AC-powered)	✓	$\sim$	$\sim^{\dagger}$	$\sim$
Intermittent queries (5–10/hr)	—	✓	$\sim^{\dagger}$	✓
Sustained agent (>20/hr)	—	×	×	✓
Battery-powered always-on	—	×	×	✓
Low-latency streaming response	✓	$\sim$	$\sim^{\dagger}$	×

5.4 Limitations and Threats to Validity

Several aspects of the experimental design limit the generalisability of these findings. The quantisation formats differ across platforms: Q4_0, q4f16_2, and GPTQ Int4, as each framework exposes different compression schemes. While all formats are nominally 4-bit, differences in grouping and calibration may introduce confounding variation in output quality and throughput. More broadly, each platform uses a different inference framework (vLLM, MLC-LLM, MLX, hailo-ollama), meaning that observed performance differences reflect the combined effect of hardware *and* software rather than hardware alone. The S24 Ultra's anomalously high TTFT, for example, may be largely framework-driven.

The study evaluates a single model (Qwen 2.5 1.5B) with a single long-form generation prompt. Shorter outputs, smaller models, or conversational prompts may exhibit different thermal trajectories. Token generation lengths also differ across platforms (564–1789 tokens), which affects thermal load duration and complicates direct comparison of thermal behaviour.

The Android data is severely limited: only 5 valid iterations were collected before an OS-enforced GPU frequency floor terminated the benchmark, and power figures are excluded due to Battery Manager API measurement unreliability. iOS does not expose per-component power draw, precluding energy-per-token comparison. The RTX 4050 was benchmarked on battery, and results represent battery-throttled performance rather than the GPU's full capability. Finally, only one unit per platform was tested; device-to-device variability in thermal performance is not characterised.

5.5 Conclusion and Future Work

Thermal throttling is the primary constraint for mobile LLM inference. The iPhone 16 Pro loses 44% of its throughput within two inferences and sustains that degradation for 65% of all test iterations. The S24 Ultra suffers a hard OS-enforced GPU frequency floor that terminates inference entirely after 6 iterations. These findings suggest that current smartphones are unsuitable for always-on agent workloads and are better suited to intermittent queries at no more than 10 per hour.

Dedicated edge NPUs offer a viable alternative. The Hailo-10H sustains 6.914 tok/s at 1.87 W with a throughput CV of 0.04% and no throttling, matching the RTX 4050 in energy proportionality at 19$\times$ lower throughput. A laptop GPU provides the highest raw throughput (131.7 tok/s) but requires a power source for sustained deployment.

Future work will extend iteration counts to 100+ for long-duration thermal profiling, replace the Android Battery Manager API with a hardware current sensor for reliable power measurement, investigate duty-cycling and active cooling strategies for mobile deployment, evaluate the impact of quantisation format unification across platforms, and assess additional models and prompt types to broaden generalisability.

References

[1] Qwen Team. (2024). Qwen2.5 technical report. arXiv preprint arXiv:2412.15115.

[2] Touvron, H., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.

[3] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR 2023*.

[4] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2023). AWQ: Activation-aware weight quantization for LLM compression and acceleration. arXiv preprint arXiv:2306.00978.

[5] Qualcomm. (2023). *Snapdragon 8 Gen 3 mobile platform product brief*. Qualcomm Technologies, Inc.

[6] Apple. (2024). *Core ML framework documentation*. Apple Developer Documentation.

[7] Reddi, V. J., et al. (2020). MLPerf inference benchmark. *Proc. ISCA*, pp. 446–459. IEEE.

[8] Ignatov, A., et al. (2018). AI benchmark: All about deep learning on smartphones. *Proc. ECCV Workshops*, pp. 318–328. Springer.

[9] Laskaridis, S., Katevas, K., Minto, L., & Haddadi, H. (2024). MELTing point: Mobile evaluation of language transformers. arXiv preprint arXiv:2403.12844.

[10] Xiao, J., Huang, Q., Chen, X., & Tian, C. (2024). Understanding large language models in your pockets: Performance study on COTS mobile devices. arXiv preprint arXiv:2410.03613.

[11] MLC Team. (2023). *MLC-LLM* [Computer software]. GitHub. `https://github.com/mlc-ai/mlc-llm`

[12] Gerganov, G. (2023). *llama.cpp* [Computer software]. GitHub. `https://github.com/ggerganov/llama.cpp`

[13] Primate Labs. (2024). *Geekbench ML benchmark documentation*. `https://www.geekbench.com/ml/`

[14] Zhang, Z., Dash, P., Hu, Y. C., Xu, Q., Li, J., & Guan, H. (2025). Dissecting the impact of mobile DVFS governors affect on-device LLM inference performance and energy efficiency. arXiv preprint arXiv:2507.02135.

[15] Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., & Zhang, C. (2023). FlexGen: High-throughput generative inference of large language models with a single GPU. *Proc. ICML 2023*.

[16] Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., & Dean, J. (2023). Efficiently scaling transformer inference. *Proc. MLSys 2023*.

[17] Shi, L., Zhang, Z., Dong, B., Xiao, Y., Li, T., Wang, J., & Wen, M. (2024). Transformer-Lite: High-efficiency deployment of large language models on mobile phone GPUs. arXiv preprint arXiv:2403.20041.

[18] Zhang, H., & Huang, J. (2025). Challenging GPU dominance: When CPUs outperform for on-device LLM inference. arXiv preprint arXiv:2505.06461.